

Como esta armado el proyecto

Estructura del código y funcionamiento del reconocimiento facial - Sistema de Asistencias, CENT35

Cómo está armado el proyecto

Sistema de Asistencias Digital con Reconocimiento Facial *Prácticas Profesionalizantes III -- CENT35, Tierra del Fuego*

Este documento explica **cómo se organiza el código** y **cómo funciona el reconocimiento facial**, sin entrar todavía en el detalle línea por línea. La idea es que después de leerlo puedas abrir cualquier archivo del proyecto y saber de antemano qué vas a encontrar adentro.

Se lee de arriba hacia abajo: primero el mapa general, después la organización de las carpetas, y al final el reconocimiento facial, que es la parte más específica.

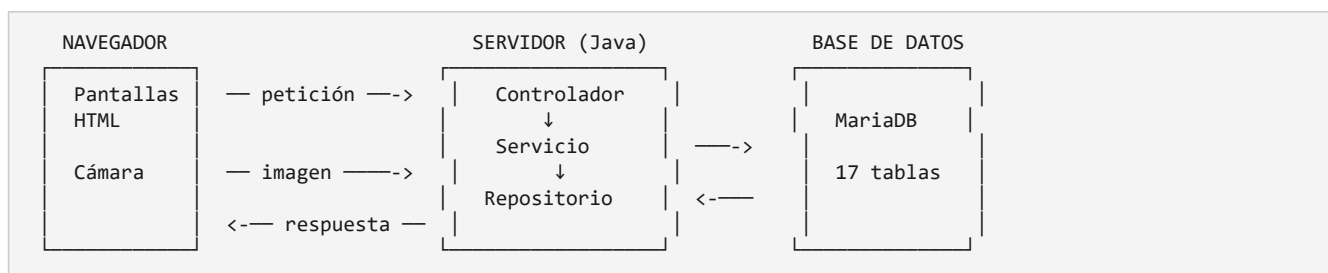
1. Qué hace el sistema, en una frase

Una cámara identifica al docente que se para frente a ella, el sistema averigua qué clase está dando en ese momento y le registra la asistencia, sin que nadie tenga que firmar nada.

Todo lo demás del proyecto existe para que eso sea posible y confiable: cargar los docentes, cargar sus horarios, obtener su consentimiento, entrenar su modelo facial, y poder revisar y corregir después lo que el sistema registró.

2. El mapa general

El proyecto es una **aplicación web que corre en un solo programa**. No hay servicios separados ni aplicación móvil: un único proceso Java atiende las páginas y hace el reconocimiento facial.



Por qué un solo programa y no varios. Es un sistema para una institución educativa que se despliega en una máquina de escritorio. Partirlo en servicios separados agregaría complejidad de despliegue sin resolver ningún problema que el proyecto tenga hoy.

Las tecnologías, y para qué está cada una

Pieza	Para qué
Java 21 + Spring Boot	El lenguaje y el armazón de la aplicación
Thymeleaf	Arma las páginas HTML en el servidor
MariaDB	Guarda todo: docentes, horarios, asistencias, modelos faciales
Flyway	Lleva el control de los cambios en la base, versionados

Pieza	Para qué
JavaCV / OpenCV	Detecta y reconoce rostros
Spring Security	Login, contraseñas y permisos

3. Cómo se organizan las carpetas de código

Todo el código Java vive en `src/main/java/edu/cent35/asistencias/` y está dividido en **seis carpetas, una por tipo de responsabilidad**.

```
edu/cent35/asistencias/
├── controller/    19 archivos  <- reciben las peticiones del navegador
├── service/       26 archivos  <- la lógica: las reglas del negocio
├── repository/    15 archivos  <- hablan con la base de datos
├── model/         25 archivos  <- las cosas del dominio: Docente, Asistencia...
├── dto/           33 archivos  <- datos que viajan entre pantalla y servidor
└── config/        11 archivos  <- seguridad, multi-institución, arranque
```

En total, unas **10.800 líneas** de Java, más 33 pantallas, 8 archivos de JavaScript y 9 migraciones de base de datos.

Qué va en cada carpeta

`controller/` -- **la puerta de entrada**. Recibe la petición del navegador, se la pasa a un servicio y devuelve la pantalla o el JSON. No decide nada: si ves una regla de negocio en un controlador, está en el lugar equivocado.

`service/` -- **donde vive el negocio**. Es la carpeta más importante. Acá está todo lo que el sistema *sabe*: que no se puede dar de baja a un docente que todavía dicta materias, que una asistencia después de la tolerancia es TARDE, que sin consentimiento no se registra un rostro.

`repository/` -- **el acceso a los datos**. Son interfaces: se declara el nombre del método y Spring Data escribe la consulta SQL. `findByDniAndInstitucionId` se traduce solo.

`model/` -- **las cosas del dominio**. Cada archivo se corresponde con una tabla (`Docente`, `Asistencia`, `Horario`) o con un conjunto cerrado de valores (`EstadoAsistencia`, que solo puede ser PRESENTE, TARDE o AUSENTE).

`dto/` -- **lo que viaja**. Objetos de transporte entre las pantallas y el servidor. Existen para **no exponer las entidades directamente**: el listado de usuarios usa un DTO que sencillamente no tiene el campo de la contraseña, así que no hay forma de que se filtre a la vista por descuido.

`config/` -- **el armazón**. Seguridad, separación entre instituciones, tareas programadas. Es la carpeta que menos se toca y la que más conviene entender.

Por qué está organizado por tipo y no por tema

Una alternativa habría sido agrupar por dominio: una carpeta `docente/` con su controlador, su servicio y su repositorio adentro, otra `asistencia/`, y así.

Se eligió agrupar por tipo porque **para un proyecto de este tamaño es más fácil de recorrer**: cuando buscás una regla de negocio sabés que está en `service/`, sin tener que adivinar a qué dominio pertenece. Los casos que cruzan varios dominios --el pase de asistencia toca docentes, horarios, comisiones y modelos faciales a la vez-- no tienen una carpeta obvia en el otro esquema.

Las otras carpetas

```
src/main/resources/
├── templates/      33 pantallas HTML (Thymeleaf)
├── static/
│   ├── css/       una sola hoja de estilos
│   └── js/        8 archivos, uno por comportamiento
├── db/migration/   9 migraciones, V001 a V009
└── opencv/        el clasificador de rostros
```

Sobre las migraciones: cada cambio en la base es un archivo numerado que **nunca se edita después de aplicarse**. Flyway guarda una huella de cada uno; si se modifica un archivo ya aplicado, la aplicación se niega a arrancar. Por eso, para eliminar una tabla creada en V001 se agrega una V009 que la borra, en vez de tocar V001.

4. El recorrido de una petición

Vale la pena seguir un caso completo, porque el mismo recorrido se repite en todo el sistema.

Alguien abre el listado de docentes:

1. El navegador pide GET /docentes.
2. Antes de llegar al controlador pasa por dos **filtros**: uno averigua a qué institución pertenece quien está usando el sistema, y otro comprueba que haya verificado su correo.
3. `DocenteController` recibe la petición y le pide la lista a `DocenteService`.
4. `DocenteService` aplica las reglas y le pide los datos a `DocenteRepository`.
5. El repositorio consulta MariaDB. **La consulta se filtra sola por institución** (ver más abajo).
6. Los datos vuelven convertidos a DTO, y Thymeleaf arma el HTML.

La separación entre instituciones

El sistema está preparado para que **varias instituciones lo usen sin verse entre sí**. Cada tabla tiene una columna que indica a qué institución pertenece cada fila.

Lo importante es que **el filtrado no depende de que el programador se acuerde de escribirlo**. Hay tres defensas superpuestas:

1. Un filtro automático que se aplica a toda consulta.
2. En las consultas escritas a mano, la condición va explícita.
3. Los servicios comprueban, al buscar por identificador, que el registro sea de la institución correcta; si no, responden "no encontrado" en vez de "no tenés permiso" --que ya sería confirmar que ese registro existe.

Hay pruebas automáticas que verifican las tres capas por separado.

5. El reconocimiento facial

Esta es la parte específica del proyecto, y la que conviene entender mejor.

La idea de fondo

El sistema **no guarda fotos**. Nunca. Lo que guarda es un **modelo matemático** entrenado a partir de varias imágenes del rostro, del cual no se puede reconstruir la cara original.

Eso no es una decisión estética: los datos biométricos son **datos sensibles** según la Ley 25.326, y guardar fotografías sería tratar un dato sensible sin necesidad, además de crear un riesgo que no hace falta correr.

El algoritmo: LBPH, y por qué

LBPH (Local Binary Patterns Histograms) funciona así, explicado sin fórmulas:

Toma la imagen del rostro en escala de grises y la recorre píxel por píxel. Para cada uno, compara su brillo con el de sus ocho vecinos y anota un patrón de "más claro / más oscuro". Después divide la imagen en una grilla y cuenta cuántas veces aparece cada patrón en cada celda. **Ese conteo es el modelo.**

Reconocer a alguien es hacer lo mismo con la imagen nueva y medir qué tan distintos son los dos conteos. Cuanto menor la diferencia, más se parecen.

Ventaja	Límite
Corre local, sin enviar nada a internet	Sensible a los cambios de iluminación
No necesita entrenamiento previo con miles de caras	Menos preciso que las redes neuronales modernas
Viene incluido en OpenCV, que es libre	Tolera mal los cambios grandes de pose

Por qué se eligió igual: el proyecto exige herramientas de código abierto y que el reconocimiento corra en Java, sin servicios externos --enviar rostros a un servicio de terceros sería una transferencia internacional de datos sensibles--. LBPH cumple, funciona con pocas imágenes por persona y es explicable, que para un trabajo académico importa. La sensibilidad a la luz es su debilidad conocida, y buena parte de las decisiones del sistema existen para compensarla.

Las dos mitades: registrar y reconocer

El reconocimiento facial son en realidad **dos flujos distintos** que casi no comparten código.

REGISTRAR (una vez por docente)	RECONOCER (cada vez que pasa)
1. Consentimiento firmado 2. Captura guiada: 5 poses 3. Se mide la calidad de c/captura 4. Se entrena el modelo 5. Se cifra y se guarda	1. Llega un cuadro de la cámara 2. Se detecta el rostro 3. Se compara contra TODOS los modelos 4. Gana el más parecido, si baja del umbral 5. La identidad se sostiene 3 segundos 6. Se registra la asistencia

Los servicios que intervienen

Cada uno hace una sola cosa. En conjunto son unas 1.150 líneas.

Servicio	Qué hace	Líneas
DeteccionRostroService	Encuentra el rostro dentro de la imagen y lo recorta	215
CalidadCapturaService	Mide si la captura sirve: nitidez, luz, encuadre	145
ModeloFacialService	Orquesta el registro: valida, entrena, cifra, guarda	276
MotorLbphService	El trato directo con OpenCV: entrenar y serializar	128
CifradoBiometricoService	Cifra y descifra el modelo	35
IdentificacionFacialService	Compara un rostro contra los modelos guardados	189
VentanaConfirmacionService	Exige que la identidad se sostenga antes de marcar	69
PaseAsistenciaService	Coordina todo el pase de asistencia	94

Flujo 1: registrar el rostro de un docente

Paso 0 -- el consentimiento. Sin consentimiento vigente el sistema **no deja registrar el rostro**. No es un aviso que se puede saltar: es una condición que se verifica en el servidor.

Paso 1 -- la captura guiada. La pantalla pide cinco poses (de frente, apenas a la izquierda, apenas a la derecha, mentón arriba, un paso más cerca) y toma tres capturas de cada una. No hay botón de grabar: la cámara captura sola cuando la imagen cumple los criterios.

Por qué cinco poses y no una. Antes el sistema grababa 30 segundos seguidos, y alguien quieto frente a la cámara produce veinte fotos casi idénticas. Un modelo entrenado así aprende **una sola pose**, y después falla en cuanto la persona inclina la cabeza. Las poses distintas son las que le dan tolerancia.

Por qué los giros son suaves. El detector busca rostros **de frente**. Ante un perfil marcado no encuentra la cara, o encuentra un recorte mal alineado que ensucia el modelo en vez de mejorarlo.

Paso 2 -- la calidad. De cada captura se mide:

- **Nitidez** -- si la imagen está movida, pierde los bordes finos.
- **Brillo y contraste** -- detecta la oscuridad y el contraluz.
- **Encuadre** -- qué porcentaje del cuadro ocupa la cara.

Si algo no cumple, la pantalla dice **qué corregir**: *acercate, quedate quieto, hay poca luz*.

Paso 3 -- la variedad. Antes de entrenar se comparan las capturas entre sí. Si son casi idénticas, se descartan por repetidas. Es lo que impide completar las cinco etapas sin haberse movido.

Paso 4 -- entrenar, comprimir y cifrar.

```
15 recortes  -->  entrenar LBPH  -->  comprimir  -->  cifrar  -->  guardar
                                   (gzip)           (AES)
```

La compresión no es un detalle menor: el modelo que produce OpenCV es un archivo de texto que pesa varios MB, y sin comprimir el guardado excede el tamaño máximo que acepta la base.

Paso 5 -- el histórico. Si el docente ya tenía un modelo, el anterior **no se borra**: se da de baja y queda como histórico. La única operación que borra de verdad es la supresión por derecho ARCO, que es un derecho de la persona sobre sus datos.

Flujo 2: reconocer y marcar

El navegador manda **un cuadro por segundo** mientras el pase está activo. Con cada uno:

Paso 1 -- detectar. Se busca el rostro. Si no hay ninguno, o hay más de uno, se responde y no se sigue.

Paso 2 -- normalizar. El recorte se pasa a gris, se le empareja la iluminación y se escala a un tamaño fijo. Esto es lo que permite comparar dos fotos tomadas a distinta distancia.

Paso 3 -- comparar contra todos. Se recorren los modelos activos **de esa institución** y se calcula la distancia contra cada uno.

Acá hay un detalle importante: se guarda **el mejor y también el segundo mejor**. La diferencia entre ambos es el **margen**, y es lo único que permite ver si el sistema estuvo cerca de confundir a dos personas. Un acierto con margen de 3 puntos está a un cuadro de convertirse en un error, y mirando solo la mejor distancia eso no se nota.

Paso 4 -- el umbral. Si la mejor distancia supera el umbral configurado, se responde "no reconocido". Es preferible no reconocer a alguien que reconocerlo mal.

Paso 5 -- sostener la identidad tres segundos. El paso más reciente, y el que resuelve un problema real: **cuando cambia la iluminación, el reconocimiento oscila** entre dos personas parecidas.

Antes se marcaba con un solo cuadro, así que el primero que saliera ganador escribía la asistencia. Ahora se exige que **el mismo docente se sostenga tres segundos**, y si en el medio aparece otro, la cuenta vuelve a cero. Así, dos personas parecidas alternándose no confirman a ninguna, en lugar de confirmar a cualquiera.

Por qué esto importa tanto. Los dos errores posibles no cuestan lo mismo. Una marca que falta se arregla con la carga manual, que existe y deja constancia. Una marca **equivocada** queda asentada como un hecho: alguien figura habiendo dado una clase que no dio, y para corregirlo alguien tiene primero que darse cuenta.

Paso 6 -- registrar la asistencia. Confirmada la identidad, el sistema busca qué clase tiene ese docente en ese momento, decide si llegó dentro de la tolerancia (PRESENTE) o después (TARDE), y guarda la marca.

El registro es **idempotente**: si la misma persona pasa dos veces por la misma clase, la segunda no crea una fila nueva. Hay una restricción en la base que lo garantiza aunque el código fallara.

El cache, y por qué existe

Descifrar y descomprimir un modelo lleva tiempo. Si hubiera que hacerlo para cada docente en cada cuadro, con un cuadro por segundo, el sistema no daría abasto.

Por eso los modelos se mantienen **cargados en memoria** una vez descifrados. El cache se sincroniza en cada identificación: carga los que aparecieron y descarta los que ya no están activos.

Esto tiene una consecuencia de seguridad que está resuelta explícitamente: cuando alguien ejerce su derecho a que le borren los datos biométricos, **no alcanza con borrarlos de la base** -- hay que sacarlos también del cache, o el sistema seguiría reconociéndolo desde memoria. El código lo hace, y en ese orden.

Qué protege los datos biométricos

Medida	Qué evita
No se guardan fotos	Que una filtración exponga imágenes de las personas
El modelo se guarda cifrado	Que sirva de algo si alguien copia la base
Consentimiento obligatorio	Tratar datos sensibles sin autorización
Registro de IP y navegador al consentir	No poder acreditar después que el consentimiento existió
Borrado real al ejercer el derecho	Que quede el dato "dado de baja" pero presente
Las asistencias sobreviven al borrado	Perder el registro administrativo de la institución

Lo que el sistema todavía no hace

Conviene decirlo con claridad, porque es la primera pregunta que aparece:

Hoy el sistema acepta una fotografía sostenida frente a la cámara. No distingue una persona presente de su imagen. La detección de vivacidad está identificada como pendiente y el control que la compensa mientras tanto es que el pase lo opera una persona que está mirando.

6. Cómo seguir

Para entrar en el detalle del código, en orden de dificultad:

1. `DocenteService` -- un servicio común, con las reglas de negocio típicas. Es el mejor punto de partida.
2. `PaseAsistenciaService` -- corto, y coordina todo el flujo del pase. Se lee en cinco minutos y da el panorama.
3. `IdentificacionFacialService` -- el corazón del reconocimiento.
4. `config/` -- seguridad y separación entre instituciones. Lo más abstracto, y lo que conviene dejar para el final.

Cada archivo empieza con un comentario de dos oraciones que dice para qué existe, y cada función tiene una línea que explica qué hace.

Las decisiones están documentadas aparte. En `docs/4-arquitectura/adr/` hay trece registros, uno por decisión no obvia, cada uno con las alternativas que se descartaron y por qué. Cuando algo del código parezca raro, es probable que ahí esté la explicación.